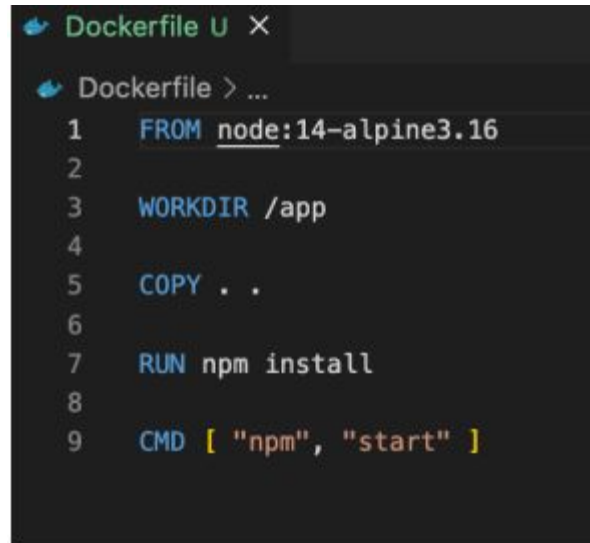


Вступ до Dockerfiles

Розуміння Dockerfile

- Dockerfile – це текстовий файл, який містить інструкції для автоматизації процесу створення образу Docker. Він служить планом для складання всіх компонентів, необхідних для запуску програми в контейнері Docker. По суті, Dockerfile окреслює кроки, необхідні для налаштування середовища в контейнері, включаючи встановлення залежностей, налаштування параметрів та виконання команд.
- Основне призначення Dockerfile – полегшити відтворюваність та перенесення програм у різних середовищах. Визначаючи точні кроки, необхідні для створення образу, Dockerfile забезпечують узгодженість та надійність процесу розгортання. Крім того, Dockerfile пропагують використання принципів інфраструктури як коду (IaC), дозволяючи розробникам контролювати версії та керувати середовищами своїх програм разом з їхньою кодовою базою.



```
Dockerfile U X
Dockerfile > ...
1 FROM node:14-alpine3.16
2
3 WORKDIR /app
4
5 COPY . .
6
7 RUN npm install
8
9 CMD [ "npm", "start" ]
```

Переваги використання Dockerfiles

- **Автоматизація:** Dockerfiles автоматизує процес створення образів Docker, усуваючи необхідність ручного втручання.
- **Узгодженість:** За допомогою Dockerfiles розробники можуть забезпечити узгодженість процесу розгортання, вказавши точні залежності та налаштування, необхідні для їхньої програми.
- **Контроль версій:** Dockerfiles можна контролювати за версіями разом із кодом програми, що дозволяє розробникам відстежувати зміни та ефективно керувати оновленнями.
- **Масштабованість:** Dockerfiles дозволяє масштабоване розгортання програм, забезпечуючи легке та ефективне рішення для контейнеризації.

Ключові компоненти Dockerfile

- **FROM:** Інструкція FROM вказує базовий образ, на основі якого буде створено новий образ.
- **RUN:** Інструкція RUN виконує команди в образі Docker під час процесу збірки.
- **CMD:** Інструкція CMD визначає команду за замовчуванням, яка виконується під час запуску контейнера Docker.
- **COPY/ADD:** Інструкції COPY та ADD копіюють файли та каталоги з хост-комп'ютера в образ Docker.
- **ENV:** Інструкція ENV встановлює змінні середовища в образі Docker.
- **EXPOSE:** Інструкція EXPOSE надає доступ до певних портів на контейнері Docker, щоб дозволити зв'язок з іншими контейнерами або зовнішніми службами.
- **WORKDIR:** Інструкція WORKDIR встановлює робочий каталог для наступних інструкцій у Dockerfile.

Інструкція FROM

Інструкція FROM у Dockerfile закладає основу для створення образів Docker. По суті, вона визначає початкову точку, що диктує середовище, в якому працює програма. Ця інструкція є життєво важливою, оскільки вона забезпечує узгодженість у різних середовищах розробки та розгортання. Вказуючи базовий образ, розробники інкапсулюють усі необхідні залежності та конфігурації, спрощуючи процес розгортання. Крім того, інструкція FROM забезпечує контроль версій, що дозволяє розробникам відстежувати зміни та забезпечувати відтворюваність. Безпека також підвищується, оскільки розробники можуть вибирати надійні базові образи та регулярно оновлювати їх, щоб виправляти вразливості. Загалом, інструкція FROM є основою створення образів Docker, впливаючи на такі фактори, як сумісність, безпека та зручність обслуговування.

Інструкція RUN

Інструкція RUN у Dockerfiles дозволяє виконувати команди в образі Docker під час процесу збірки. Ця інструкція є ключовою для встановлення пакетів, налаштування параметрів та виконання завдань налаштування, необхідних для програми. Використовуючи інструкцію RUN, розробники автоматизують процес збірки, зменшуючи ручне втручання та мінімізуючи помилки. Крім того, ця інструкція дозволяє налаштовувати образ Docker, дозволяючи розробникам адаптувати середовище до конкретних потреб програми. Від встановлення залежностей до виконання скриптів, інструкція RUN дозволяє розробникам оптимізувати процес створення образу Docker, забезпечуючи ефективність та надійність робочих процесів розгортання.

Інструкція CMD

- Інструкція CMD у Dockerfiles є критичним елементом, який визначає основну команду, що виконується під час запуску контейнера. На відміну від інструкції RUN, яка працює під час фази збірки, CMD вступає в дію під час виконання, визначаючи початкову поведінку контейнера.
- По суті, CMD встановлює дію за замовчуванням для контейнера, зазвичай представляючи основну програму або службу, яку він розміщує. Ця інструкція спрощує процес розгортання, автоматизуючи процедуру запуску, гарантуючи, що контейнери запускаються з бажаною функціональністю без ручного втручання.

Інструкція LABEL

- Інструкція LABEL у Dockerfiles відіграє вирішальну роль у анотуванні образів Docker метаданими. Ці метадані надають ключову інформацію про образ, таку як номери версій, дані про розробника та описові теги.
- Використовуючи інструкцію LABEL, розробники покращують керованість та відстеження образів Docker протягом їхнього життєвого циклу. Мітки можуть передавати різні типи інформації, включаючи інформацію про ліцензування, URL-адреси проектів та примітки до випуску, що полегшує розуміння призначення та використання образу.

Інструкція EXPOSE

- Інструкція EXPOSE в Dockerfiles відіграє життєво важливу роль в інформуванні Docker про те, які мережеві порти має відкривати контейнер. Ця інструкція фактично не публікує порти; натомість вона служить документацією для розробників та операторів щодо передбачуваних мережевих підключень.
- Вказуючи порти за допомогою інструкції EXPOSE, розробники уточнюють мережеві вимоги своїх контейнерних додатків. Ця документація особливо цінна під час співпраці з командами або розгортання додатків у різних середовищах.
- Хоча інструкція EXPOSE відкриває порти, вона не робить їх автоматично доступними ззовні контейнера. Щоб забезпечити зовнішній доступ до відкритих портів, потрібна додаткова конфігурація, така як зіставлення портів під час виконання контейнера.

Інструкція ENV

- Інструкція ENV у Dockerfiles відіграє важливу роль у налаштуванні змінних середовища в образі Docker. Змінні середовища – це динамічні значення, які можуть впливати на поведінку програмних застосунків та служб, що працюють у контейнері.
- Використовуючи інструкцію ENV, розробники можуть визначати ключові параметри, такі як рядки підключення до бази даних, кінцеві точки API та налаштування, специфічні для програми, безпосередньо в Dockerfile. Це спрощує процес налаштування та покращує переносимість образів Docker у різних середовищах.
- Крім того, до змінних середовища, встановлених за допомогою інструкції ENV, можна отримати доступ за допомогою наступних інструкцій у Dockerfile, і вони зберігаються в контейнері під час виконання. Це забезпечує узгодженість та відтворюваність поведінки контейнерних застосунків у різних сценаріях розгортання.

Інструкція ARG

- Інструкція ARG у Dockerfiles дозволяє розробникам визначати змінні часу збірки, які можна передавати в процес збірки Docker. Ці змінні забезпечують гнучкість та можливості налаштування під час створення образу, дозволяючи розробникам параметризувати свої Dockerfiles та робити їх більш адаптованими до різних сценаріїв.
- Змінні ARG передаються команді збірки Docker за допомогою прапорця -build-arg, що дозволяє розробникам перевизначати їхні значення за замовчуванням під час збірки. Це забезпечує більший контроль та гнучкість, особливо в автоматизованих конвеєрах збірки або сценаріях, де для різних середовищ потрібні різні конфігурації.

Інструкції ARG проти ENV

ARG (Змінні часу збірки):

- Використовується для передачі аргументів під час збірки.
- Значення доступні лише під час процесу збірки та не зберігаються у фінальному образі.
- Корисно для надання динамічних значень Dockerfile, таких як номери версій або шляхи до файлів.
- Синтаксис: ARG variable_name=default_value

ENV (Змінні часу виконання):

- Встановлює змінні середовища, доступні під час виконання контейнера.
- Значення зберігаються в кінцевому образі та можуть бути перевизначені під час створення екземпляра контейнера.
- Часто використовується для налаштування параметрів програми або визначення поведінки під час виконання.
- Синтаксис: ENV variable_name=value

Інструкція ADD

- Інструкція ADD у Dockerfiles спрощує додавання файлів та каталогів з хост-машини до образу Docker. Ця інструкція є універсальною, що дозволяє розробникам додавати широкий спектр ресурсів, включаючи код програми, файли конфігурації та статичні ресурси.
- Використовуючи інструкцію ADD, розробники спрощують процес створення образу Docker, гарантуючи, що всі необхідні компоненти легкодоступні в контейнері. Це спрощує процес розгортання та зменшує залежність від зовнішніх ресурсів, підвищуючи портативність та самодостатність образів Docker.

Інструкція COPY

- Інструкція COPY в Dockerfiles спрощує передачу файлів та каталогів з хост-машини в образ Docker. Ця інструкція є фундаментальною для включення коду програми, файлів конфігурації та інших ресурсів, необхідних для контейнеризованої програми.
- Використовуючи інструкцію COPY, розробники гарантують, що всі необхідні ресурси включені до образу Docker, оптимізуючи процес розгортання та зменшуючи залежність від зовнішніх ресурсів. Крім того, COPY забезпечує прозорість та контроль над процесом передачі файлів, підвищуючи відтворюваність та надійність збірок образів Docker.

Інструкції ДОДАТИ проти КОПІЮВАННЯ

COPY:

- COPY — це простий у використанні інструмент, який в першу чергу призначений для копіювання локальних файлів і каталогів у образ.
- Він пропонує прозорість і простоту, що робить його безпечнішим вибором для більшості завдань передачі файлів.
- COPY не підтримує URL-адреси та не розпаковує стиснуті файли автоматично, зосереджуючись виключно на копіюванні файлів з хост-комп'ютера.

ADD:

- ADD є більш універсальним і пропонує додаткові функції, окрім простого копіювання файлів.
- Окрім локальних файлів і каталогів, ADD підтримує URL-адреси та може автоматично розпаковувати стиснуті файли.
- Хоча ADD забезпечує гнучкість, він несе потенційні ризики для безпеки під час використання з URL-адресами, оскільки може ненавмисно створювати вразливості, якщо джерело ненадійне.

Інструкція ENTRYPOINT

- Інструкція ENTRYPOINT у Dockerfiles встановлює основну команду, яка буде виконана під час запуску контейнера. На відміну від інструкції CMD, яку можна перевизначити під час виконання, ENTRYPOINT визначає незмінну початкову точку для контейнера.
- Ця інструкція зазвичай використовується для визначення головного виконуваного файлу або процесу, який запускатиме контейнер. Вона забезпечує узгодженість та передбачуваність поведінки контейнера, гарантуючи, що важливі служби або програми завжди запускаються належним чином.
- Використовуючи інструкцію ENTRYPOINT, розробники встановлюють чітку точку входу для контейнеризованих програм, підвищуючи їхню зручність використання та підтримку. Ця інструкція особливо цінна в сценаріях, коли контейнеризована програма вимагає певної точки входу або послідовності ініціалізації.

CMD проти ENTRYPOINT

CMD:

- CMD встановлює команду та/або параметри за замовчуванням, які можна перевизначити під час виконання контейнера.
- Його часто використовують для визначення головного виконуваного файлу або процесу, який контейнер запускає за замовчуванням.
- CMD є гнучким і дозволяє користувачам вказувати альтернативні команди під час запуску контейнера.

ENTRYPOINT:

- ENTRYPOINT встановлює основну команду, яка завжди виконуватиметься під час запуску контейнера.
- Вона забезпечує незмінну початкову точку для контейнера і не може бути перевизначена під час виконання.
- ENTRYPOINT зазвичай використовується для визначення головного виконуваного файлу або процесу, який контейнер повинен запускати.

Інструкція VOLUME

- Інструкція VOLUME в Dockerfiles спрощує створення точок монтування в контейнері, надаючи доступ до каталогів на хост-машині або інших контейнерах. Ця інструкція є важливою для керування постійними даними та обміну файлами між контейнером та його середовищем.
- Використовуючи інструкцію VOLUME, розробники можуть вказувати каталоги в контейнері, які мають зберігатися після завершення життєвого циклу контейнера. Це забезпечує безперешкодний обмін даними та їх зберігання між контейнером та хост-машиною або іншими контейнерами в одній мережі Docker.

Інструкція USER

- Інструкція USER у Dockerfiles встановлює контекст користувача для контейнера, визначаючи обліковий запис користувача, під яким працює контейнеризований додаток. Ця інструкція є критично важливою для підвищення безпеки та керування дозволами в середовищі контейнера.
- Вказуючи користувача без прав root за допомогою інструкції USER, розробники зменшують потенційні ризики безпеки, пов'язані із запуском контейнерів від імені користувача root. Ця практика дотримується принципу найменших привілеїв, зменшуючи вплив вразливостей безпеки та несанкціонованого доступу до конфіденційних системних ресурсів.

Інструкція WORKDIR

- Інструкція WORKDIR у Dockerfiles визначає робочий каталог для контейнера, вказуючи шлях до каталогу, де команди будуть виконуватися за замовчуванням. Ця інструкція спрощує операції з файлами та виконання команд у контейнері, підвищуючи ефективність та читабельність Dockerfiles.
- Використовуючи інструкцію WORKDIR, розробники можуть встановити узгоджену структуру каталогів для своїх контейнерних програм, гарантуючи правильне визначення відносних шляхів. Це сприяє портативності та простоті використання, оскільки розробники можуть покладатися на попередньо визначену структуру каталогів незалежно від середовища хоста.

Інструкція ONBUILD

- Інструкція ONBUILD у Dockerfiles визначає тригери, які виконуються, коли образ використовується як основа для іншої збірки. Ці тригери дозволяють розробникам автоматизувати дії або конфігурації, які мають відбуватися в залежних образах, спрощуючи процес успадкування образів Docker.
- Коли образ з інструкціями ONBUILD використовується як основа для іншої збірки, зазначені команди додаються до Dockerfile дочірнього образу. Це дозволяє автоматично виконувати попередньо визначені дії, такі як копіювання файлів, встановлення залежностей або запуск скриптів, без зміни оригінального Dockerfile.
- Тригери ONBUILD корисні для створення образів Docker, які можна повторно використовувати та розширювати, і які можна адаптувати до різних випадків використання або середовищ. Вони надають спосіб інкапсуляції поширених кроків збірки або конфігурацій, зменшуючи дублювання та покращуючи узгодженість між кількома проектами або розгортаннями.

Найкращі практики для ефективної роботи з зображеннями

- **Об'єднання команд:** Об'єднання пов'язаних команд в одну інструкцію RUN, щоб мінімізувати кількість створюваних шарів, зменшуючи розмір образу та час збірки.
- **Використання багатоетапних збірок:** Використовуйте багатоетапні збірки, щоб відокремити залежності збірки від кінцевого образу, зменшуючи його розмір, виключаючи непотрібні інструменти збірки та артефакти.
- **Залежності кешу:** Упорядкування інструкцій Dockerfile для максимізації використання кешу, використання переваг кешованих шарів та пришвидшення наступних збірок.
- **Оптимізація COPY та ADD:** Копіювання лише необхідних файлів у образ, уникаючи великих каталогів або непотрібних файлів, та використання .dockerignore для виключення нерелевантного контенту.
- **Мінімізація розміру образу:** Видалення непотрібних файлів та залежностей, щоб зменшити розмір образу, вибір легких базових образів, таких як Alpine Linux, та уникання встановлення непотрібних пакетів.
- **Очищення:** Видалення тимчасових файлів та невикористаних залежностей на тому ж рівні, де вони створені, щоб мінімізувати загальний розмір образу.
- **Перевірка та рефакторинг:** Регулярно переглядайте та рефакторингуйте Dockerfile, щоб забезпечити дотримання найкращих практик, зберігаючи їх простими, читабельними та зручними для обслуговування.

Оптимізація кешування Dockerfile

- **Незмінні шари:** Пріоритет незмінних шарів для максимізації ефективності кешування.
- **Упорядкування інструкцій:** Розташування інструкцій від найменш змінних до найбільш змінних для кращого кешування.
- **Використання кешу:** Використовуйте зручні для кешу інструкції, такі як COPY та явні теги версій.
- **Окремі залежності:** Використовуйте багатоетапні збірки для ізоляції встановлення залежностей.
- **Оптимізація шарів:** Об'єднання команд для мінімізації створення шарів.
- **Моніторинг:** Регулярний моніторинг продуктивності збірки для виявлення областей оптимізації.

Багатоетапні збірки

```
FROM eclipse-temurin:17-jdk-alpine as builder
WORKDIR /opt/app
COPY .mvn/ .mvn
COPY mvnw pom.xml ./
RUN ./mvnw dependency:go-offline
COPY ./src ./src
RUN ./mvnw clean install

FROM eclipse-temurin:17-jre-alpine
WORKDIR /opt/app
COPY --from=builder /opt/app/target/*.jar /opt/app/*.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "/opt/app/*.jar"]
```

Багатоетапна збірка в Docker — це методологія створення образів Docker, яка передбачає поділ процесу створення образу на кілька етапів. Кожен етап виконує певні завдання, такі як компіляція програмного забезпечення, запуск тестів або розгортання програм, і може мати власний контекст виконання та залежності.

- **Оптимізація розміру:** Відкидаючи непотрібні артефакти та залежності на проміжних етапах, багатоетапні збірки призводять до менших та ефективніших кінцевих образів.
- **Покращена безпека:** Інструменти та залежності під час збірки ізольовані від кінцевого робочого образу, що зменшує потенційні вектори атак та підвищує безпеку.
- **Ефективність робочого процесу:** Завдяки багатоетапним збіркам ви можете автоматизувати складні процеси збірки в одному Dockerfile, оптимізуючи робочі процеси розробки та забезпечуючи узгодженість у різних середовищах.
- **Швидші збірки:** Усунення потреби в проміжних образах та оптимізація процесу збірки прискорюють загальний час збірки, що дозволяє пришвидшити цикли ітерацій та швидше розгорнути програми.

Найкращі практики безпеки в Dockerfiles

У Dockerfiles безпека є надзвичайно важливою для захисту контейнерних застосунків. Однією з фундаментальних практик безпеки є уникнення запуску контейнерів з правами root. Натомість використання інструкції USER дозволяє вказати користувача без прав root для виконання контейнера, тим самим зменшуючи потенційний вплив вразливостей безпеки. Крім того, критично важливо покладатися на офіційні образи Docker з надійних джерел, оскільки вони проходять ретельне тестування безпеки та обслуговування. Регулярне оновлення базових образів та залежностей є важливим для виправлення відомих вразливостей та застосування останніх виправлень безпеки. Більше того, мінімізація поверхні атаки шляхом видалення непотрібних пакетів та файлів з образів допомагає зменшити ризик потенційних експлойтів. Реалізація принципу найменших привілеїв ще більше посилює безпеку, обмежуючи дозволи контейнера лише тим, що необхідно для функціонування застосунку. Увімкнення Docker Content Trust забезпечує додатковий рівень безпеки, перевіряючи цілісність та автентичність образів Docker. Нарешті, використання інструментів сканування зображень допомагає виявляти та усувати вразливості безпеки в образах Docker перед розгортанням, забезпечуючи надійну систему безпеки для контейнерних застосунків.

Використання офіційних базових зображень

- Коли ви обираєте офіційні образи Docker як основу для ваших контейнерів, ви обираєте основу, створену та підтримувану самим Docker. Ці образи не лише регулярно оновлюються та підтримуються, але й проходять ретельне тестування та перевірку, що гарантує їхню надійність та безпеку.
- Однією з ключових переваг є надійність, яка постачається з цими образами. Ви можете покластися на них, щоб забезпечити стабільне та безпечне середовище для ваших застосунків. Крім того, офіційні образи відповідають суворим стандартам якості, що забезпечує узгодженість у різних середовищах.
- Більше того, офіційні образи мають широку підтримку спільноти та документацію. Це означає, що якщо у вас виникнуть будь-які проблеми або вам знадобиться допомога, ви, ймовірно, знайдете її у активній спільноті Docker.

Керування дозволами в Dockerfiles

- Використовуйте інструкцію USER: Вкажіть користувача без прав root з відповідними правами доступу, використовуючи інструкцію USER. Це допомагає мінімізувати ризики безпеки, пов'язані із запуском контейнерів від імені користувача root.
- Явне встановлення дозволів: Під час копіювання файлів у контейнер за допомогою інструкцій COPY або ADD явно встановіть дозволи за допомогою параметра --chown, щоб переконатися, що файли належать відповідному користувачеві.
- Розгляньте монтування томів: Під час роботи з монтуванням томів переконайтеся, що дозволи каталогу хоста налаштовані належним чином, щоб запобігти несанкціонованому доступу з контейнера.
- Уникайте дозволів 777: Уникайте використання надмірно дозвільних дозволів на файли (наприклад, 777), оскільки вони можуть створювати ризики безпеки, надаючи непотрібний доступ до файлів і каталогів.
- Дотримуйтесь принципу найменших привілеїв: Застосовуйте принцип найменших привілеїв, надаючи лише необхідні дозволи для файлів і каталогів у контейнері, мінімізуючи потенційний вплив порушень безпеки.

Впровадження Health Checks

- У Dockerfiles включення перевірок справності за допомогою інструкції HEALTHCHECK є критично важливим для підтримки справності та стабільності ваших контейнерів.
- Щоб зробити це ефективно, ви вказуєте команду у своєму Dockerfile, яку Docker періодично виконуватиме для визначення стану справності вашого контейнера. Ця команда може бути будь-чим: від запиту ping до певної кінцевої точки до запиту до бази даних або виконання власного скрипта, залежно від вимог вашої програми.
- Важливо налаштувати частоту, з якою Docker виконуватиме ці перевірки справності, використовуючи прапорець `--interval`. Це гарантує регулярний моніторинг стану справності вашого контейнера. Крім того, ви можете встановити період очікування за допомогою прапорця `--timeout`, визначаючи, як довго Docker чекатиме на відповідь, перш ніж вважати перевірку невдалою.

Використання файлів .dockerignore

- Під час створення образу Docker, Docker збирає всі файли та каталоги у вказаному каталозі контексту збірки та надсилає їх демону Docker. Цей каталог зазвичай містить Dockerfile та будь-які інші необхідні файли для збірки образу.
- Однак не всі файли в контексті збірки мають відношення до процесу створення образу. Деякі файли, такі як залежності розробки, артефакти збірки або конфіденційні файли конфігурації, можуть бути непотрібними та можуть без потреби збільшувати розмір контексту збірки.
- Саме тут на допомогу приходить файл .dockerignore. Він дозволяє вказати шаблони для файлів та каталогів, які Docker має ігнорувати під час процесу збірки. Ці шаблони мають той самий синтаксис, що й файли .gitignore, що дозволяє легко вказати, які файли або каталоги виключати, на основі їхніх імен, розширень або шляхів.

Розуміння контексту збірки в Docker

- Контекст збірки — це набір файлів і каталогів, які Docker використовує під час створення образу. Він включає Dockerfile та будь-які інші файли або каталоги, зазначені в команді build.
- Коли ви ініціюєте збірку, Docker надсилає весь контекст збірки демону Docker. Це означає, що розмір контексту збірки безпосередньо впливає на тривалість процесу збірки.
- Більший контекст збірки довше передається демону Docker і збільшує загальний час збірки. І навпаки, менший контекст збірки пришвидшує процес збірки, зменшуючи час передачі та підвищуючи загальну ефективність.
- Тому важливо ретельно керувати вмістом контексту збірки, забезпечуючи включення лише необхідних файлів і каталогів. Саме тут стають у пригоді такі інструменти, як файли .dockerignore, що дозволяють виключати нерелевантні файли та каталоги з контексту збірки.
- Оптимізуючи контекст збірки, ви можете мінімізувати час збірки, покращити продуктивність і підвищити ефективність процесу створення образу Docker.

Налагодження збірок Docker

- Перевірка Dockerfile: Ретельно перевірте Dockerfile на наявність синтаксичних помилок, відсутніх команд або залежностей.
- Перевірка контексту збірки: Переконайтеся, що всі необхідні файли включені до контексту збірки, використовуючи .dockerignore для виключення нерелевантних файлів.
- Перевірка журналів: Аналіз журналів збірки на наявність повідомлень про помилки або попереджень для виявлення проблем.
- Використовуйте --no-cache: Уникайте кешованих шарів за допомогою прапорця --no-cache для усунення проблем, пов'язаних з кешуванням.
- Інструменти налагодження: Використовуйте команди Docker, такі як docker inspect та docker history, для перевірки метаданих зображень та історії шарів.
- Інтерактивні збірки: Запускайте Docker в інтерактивному режимі (прапорець -i) для налагодження в реальному часі під час збірки.
- Ресурси спільноти: Звертайтеся за допомогою до спільноти Docker через форуми та документацію для отримання експертної поради.

Динамічна генерація Dockerfile

- **Шаблонні механізми:** Використовуйте шаблонні механізми, такі як Jinja2 або Go, для динамічної генерації Dockerfile. Ці механізми дозволяють вставляти змінні, умови та цикли в шаблони Dockerfile, що дозволяє налаштування на основі динамічних параметрів.
- **Мови сценаріїв:** Використовуйте мови сценаріїв, такі як Python, Bash або PowerShell, для програмної генерації Dockerfile. Ці сценарії можуть зчитувати файли конфігурації, змінні середовища або інші вхідні дані для динамічної генерації вмісту Dockerfile.
- **Аргументи часу збірки:** Використовуйте аргументи часу збірки (ARG) у Dockerfile для передачі динамічних значень під час процесу збірки. Це дозволяє параметризувати інструкції Dockerfile на основі вхідних даних часу збірки.
- **Багатоетапні збірки:** Використовуйте багатоетапні збірки для динамічної генерації Dockerfile на основі різних етапів процесу збірки. Кожен етап може мати власний набір інструкцій та залежностей, що дозволяє створювати складні та налаштовані Dockerfile.
- **Зовнішні інструменти:** Використовуйте зовнішні інструменти або бібліотеки, спеціально розроблені для генерації Dockerfile. Ці інструменти можуть пропонувати розширені функції та можливості для динамічної генерації Dockerfile, спрощуючи процес та підвищуючи гнучкість.

Лінтери Dockerfile

- Hadolint: Hadolint — це лінтер для Dockerfile, який зосереджений на найкращих практиках та поширених помилках.
- Dockerfilelint: Dockerfilelint — це легкий лінтер для Dockerfile, який перевіряє наявність синтаксичних помилок та потенційних проблем.
- Trivy: Trivy — це, перш за все, сканер вразливостей для образів контейнерів, але він також включає можливості лінтингу Dockerfile.
- Dockle: Dockle — це лінтер безпеки та найкращих практик для Dockerfile та образів контейнерів.
- Лінтери в IDE: Деякі інтегровані середовища розробки (IDE) пропонують вбудовану підтримку лінтингу Dockerfile.





Докеризація веб- застосунку Node.js



У першій частині цього посібника ми створимо простий вебзастосунок на Node.js, потім створимо образ Docker для цього застосунку та, нарешті, створимо контейнер з цього образу.

Docker дозволяє упакувати застосунок з його середовищем та всіма його залежностями в «коробку», яка називається контейнером. Зазвичай контейнер складається з застосунку, що працює в спрощеній до базової версії операційної системи Linux. Образ — це план для контейнера, контейнер — це запущений екземпляр образу.

Створення застосунку Node.js

Спочатку створіть новий каталог, де будуть зберігатися всі файли. У цьому каталозі створіть файл package.json, який описує ваш додаток та його залежності:

```
{
  "name": "docker_web_app",
  "version": "1.0.0",
  "description": "Node.js on Docker",
  "author": "First Last <first.last@example.com>",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.16.1"
  }
}
```

Створення застосунку Node.js

З вашим новим файлом package.json запустіть команду `npm install`. Якщо ви використовуєте npm версії 5 або пізнішої, це створить файл `package-lock.json`, який буде скопійовано до вашого образу Docker.

Потім створіть файл `server.js`, який визначає веб-застосунок за допомогою фреймворку Express.js:

```
'use strict';

const express = require('express');

// Constants
const PORT = 8080;
const HOST = '0.0.0.0';

// App
const app = express();
app.get('/', (req, res) => {
  res.send('Hello World');
});

app.listen(PORT, HOST);
console.log(`Running on http://${HOST}:${PORT}`);
```

У наступних кроках ми розглянемо, як можна запустити цей застосунок всередині контейнера Docker, використовуючи офіційний образ Docker. Спочатку вам потрібно буде створити образ Docker вашого застосунку.

Створення Dockerfile

Створіть порожній файл з назвою Dockerfile:

```
touch Dockerfile
```

Відкрийте Dockerfile у вашому улюбленому текстовому редакторі.

Перше, що нам потрібно зробити, це визначити, з якого образу ми хочемо зібрати. Тут ми використовуватимемо останню LTS (довгострокову підтримку) версію 12 для node, доступну в Docker Hub:

```
FROM node:12
```

Створення Dockerfile

Далі ми створюємо каталог для зберігання коду програми всередині зображення, це буде робочий каталог для вашої програми:

```
# Create app directory
WORKDIR /usr/src/app
```

Цей образ постачається з уже встановленими Node.js та NPM, тому наступне, що нам потрібно зробити, це встановити залежності вашої програми за допомогою бінарного файлу npm. Зверніть увагу, що якщо ви використовуєте npm версії 4 або ранішої, файл package-lock.json не

```
# Install app dependencies
# A wildcard is used to ensure both package.json AND package-lock.json are copied
# where available (npm@5+)
COPY package*.json ./

RUN npm install
# If you are building your code for production
# RUN npm ci --only=production
```

Створення Dockerfile

Зверніть увагу, що замість копіювання всього робочого каталогу ми копіюємо лише файл `package.json`. Це дозволяє нам скористатися перевагами кешованих шарів Docker. Крім того, команда `npm ci`, зазначена в коментарях, допомагає забезпечити швидші, надійніші та відтворювані збірки для виробничих середовищ.

Щоб помістити вихідний код вашої програми в образ Docker, використовуйте інструкцію `COPY`:

```
# Bundle app source
COPY . .
```

Ваша програма прив'язується до порту 8080, тому ви використовуватимете інструкцію `EXPOSE`, щоб демон docker зіставив його:

```
EXPOSE 8080
```


Створення Dockerfile

І останнє, але не менш важливе: визначте команду для запуску вашої програми за допомогою CMD, яка визначає ваше середовище виконання. Тут ми використовуємо node server.js для запуску вашого сервера:

```
CMD [ "node", "server.js" ]
```

Ваш Dockerfile тепер має виглядати так:

```
FROM node:12

# Create app directory
WORKDIR /usr/src/app

# Install app dependencies
# A wildcard is used to ensure both package.json AND package-lock.json are copied
# where available (npm@5+)
COPY package*.json ./

RUN npm install

# If you are building your code for production
# RUN npm ci --only-production

# Bundle app source
COPY . .

EXPOSE 8080

CMD [ "node", "server.js" ]
```

Файл .dockerignore

Створіть файл .dockerignore в тому ж каталозі, що й ваш Dockerfile, з таким вмістом:

```
node_modules  
npm-debug.log
```

Це запобіжить копіюванню локальних модулів та журналів налагодження на образ Docker та, можливо, перезапису модулів, встановлених у вашому образі.

Створення вашого зображення

Перейдіть до каталогу, де знаходиться ваш Dockerfile, і виконайте таку команду, щоб зібрати образ Docker. Прапорець -t дозволяє позначити образ, щоб його було легше знайти пізніше за допомогою команди docker images:

```
docker build -t <your username>/node-web-app .
```

Ваше зображення тепер буде відображено в Docker:

```
$ docker images
```

```
# Example
```

REPOSITORY	TAG	ID	CREATED
node	12	1934b0b038d1	5 days ago
<your username>/node-web-app	latest	d64d3505b0d2	1 minute ago

Запустіть зображення

Запуск образу з параметром `-d` запускає контейнер у відокремленому режимі, залишаючи його працювати у фоновому режимі. Прапорець `-p` перенаправляє публічний порт на приватний порт усередині контейнера. Запустіть образ, який ви зібрали раніше:

```
docker run -p 49160:8080 -d <your username>/node-web-app
```

Виведіть вивід вашої програми:

```
# Get container ID
$ docker ps

# Print app output
$ docker logs <container id>

# Example
Running on http://localhost:8080
```

Запустіть зображення

Якщо вам потрібно зайти всередину контейнера, ви можете скористатися командою `exec`:

```
# Enter the container  
$ docker exec -it <container id> /bin/bash
```

Тест

Щоб протестувати свій застосунок, отримайте порт вашого застосунку, який налаштував Docker:

```
$ docker ps
```

Example

ID	IMAGE	COMMAND	...	PORTS
ecce33b30ebf	<your username>/node-web-app:latest	npm start	...	49160->8080

У наведеному вище прикладі Docker зіставив порт 8080 всередині контейнера з портом 49160 на вашому комп'ютері. Тепер ви можете викликати свій застосунок за допомогою curl (встановити, якщо потрібно, за допомогою: `sudo apt-get install curl`):

```
$ curl -i localhost:49160
```

HTTP/1.1 200 OK
X-Powered-By: Express
Content-Type: text/html; charset=utf-8
Content-Length: 12
ETag: W/"c-M6tW0b/Y57lesdjQuHeB1P/qTV0"
Date: Mon, 13 Nov 2017 20:53:59 GMT
Connection: keep-alive

Hello world

Питання та відповіді